Development of a Heterogeneous Reconfigurable CGRA Cell

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian, Zúñiga Ramírez Jose Eduardo
School of Electronics Engineering
Costa Rica Institute of Technology
{davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga}@estudiantec.cr

***Abstract*—Coarse-Grained Reconfigurable Arrays (CGRAs) offer an efficient middle ground between the flexibility of FPGAs and the performance of ASICs, but most existing designs rely on homogeneous arrays of general-purpose processing elements (PEs), leaving functional units and interconnects under-utilized when the workload does not match their fixed capabilities. Heterogeneous proposals such as HETA and Plasticine improve resource usage but remain limited to either scalar or vector operations, not both. This work presents a heterogeneous CGRA cell composed of five specialized element types: routing cells with non-blocking FIFOs, distributed SRAM memory cells, a multi-precision scalar unit (int8/16/32), a float32 SIMD vector unit, and a dedicated Multiply-Accumulate (MAC) unit, organized in a 2×2 mesh with a RISC-V-facing CSR/DMA interface. The architecture is being developed in SystemC and validated through a progressive testbench flow, from single-PE operations to an end-to-end sum-reduction dataflow, before translation to RTL via Vitis HLS. As preliminary results, the regression suite comprises 19 SystemC testbenches and 139 assertions with zero failures, and an end-to-end sum-reduction benchmark executes correctly end-to-end through the RISC-V-facing CSR/DMA bridge; the arithmetic kernel was also cross-validated in Vitis HLS C-simulation with an exact match against the SystemC results. Work in progress includes full HLS synthesis and timing/resource metrics, a RISC-V (RV64GC) baseline characterization in gem5 for comparison, and broader benchmarking using MachSuite and PolyBench kernels. This paper reports the architectural proposal, methodology, and these preliminary results as an intermediate milestone toward the complete design.**

***Index Terms*—coarse-grained reconfigurable architecture (CGRA), heterogeneous CGRA, spatial CGRA, reconfigurable computing.**

I. Introduction

The reconfigurable architectures have been very successful in industry because software evolution has advanced rapidly, with diverse applications, different user needs, and scientific progress, so hardware cannot adapt at the same pace as software [1]. Special-purpose systems can suffer from rapid obsolescence, and if the price of hardware development and production is high, it becomes economically unviable [1].

Currently, architectures such as FPGAs (Field Programmable Gate Arrays) have been widely adopted in diverse applications due to their flexibility and customization capability, gaining traction in HPC (High-Performance Computing). However, FPGAs present certain limitations such as slow configuration times, long compilation times, and in cases, low operating frequencies [2].

In the recent years there has been growing interest in HPC in using architectures such as CGRAs (Coarse-Grained Reconfigurable Arrays), which offer an intermediate solution between FPGAs and ASICs (Application Specific Integrated Circuits). CGRAs are hardware architectures that, conceptually, allow silicon to be malleable and their functionality to be dynamically reconfigured [2]. CGRAs have the advantage of being one to two orders of magnitude more efficient than FPGAs and more than 2 to 3 orders of magnitude more efficient than ASICs [1].

The development of a reconfigurable CGRA cell is of great importance not only for its relevance in HPC, but also for its adoption in Machine-Learning applications, image processing, and computer vision, communications, and other areas [3]. Therefore, the objective of this work is to develop a heterogeneous CGRA with different kind of processing elements such as a scalar and vector processing cells, memory banks and a MAC (Multiply-Accumulate) unit.

II. Background and Related Work

In recent years, CGRAs have gained significant prominence as hardware accelerators due to their ability to offer an exceptional balance between software flexibility and the high performance of application-specific hardware [1], [2]. Unlike fine-grained architectures, CGRAs operate at the word or block level, which substantially reduces reconfiguration and routing overhead, making them particularly attractive for efficient edge-computing applications. The development and research of these architectures has been strongly driven by the adoption of the open-source RISC-V instruction set architecture (ISA) [4], whose extensible nature facilitates the creation of highly customized heterogeneous SoCs (Systems on Chip).

At the level of practical implementation, these architectures are the result of cutting-edge open-source efforts. Repositories such as risc-v-cgra (developed by ECASLab) investigate these integrations at the system level. OpenCGRA is a unified framework that supports the full top-to-botton design flow, this adopts a Python-based PyMTL3 framework to model the architecture and enabling the RTL generation and muti-level simulation [5] TRAM is another example which adopts a Chisel-based CGRA template to model hardware architecture with register transfer level (RTL) code generation and functional-level (FL) simulation; TRAM proposes a flexible and scalable

CGRA template composed of GPEs, IOBs and GIBs with two-dimensional (2D) island-style topology [6]. HETA is a temporal CGRA implementing a heterogeneous architecture with a Bayesian optimization, this CGRA is a 2D island style layout build from four type of blocks: GPE (Generic Processing Element), GIB (General Interconnect Block), LSU (Load/Store Unit) and IOB (I/O Block) [7].

For this type of system to reach its full potential, the internal design of the individual reconfigurable cell (PE) and its associated compilation tools are critical [8]. They highlight the need for context-memory-aware mapping to ensure energy-efficient acceleration, emphasizing that data-access latency dictates overall performance. Within this framework, the design and development of a new reconfigurable CGRA cell is grounded in the need to optimize local interconnects, deterministic access to intermediate operands, and the versatility of its cycle-level operations — challenges that remain areas of active exploration at the frontier of heterogeneous accelerator design.

III. Analysis of Existing Solutions (SOTA)

Nowadays there are different types of CGRAs, you can classify them by their elements in heterogeneous or homogeneous, also depending on how you use structure the hardware and the handling of the data, they can be classified as temporal or spatial. Examples of homogeneous CGRAs are the HyCUBE, DySER, TRAM and OpenCGRA [5], [6], [9] which have an array of homogeneous or equal PEs, while examples of heterogeneous CGRAs are Plasticine and HETA [7], [10] which combine different PE.

CGRA systems are no longer seen as an isolated element; today they are presented as a heterogeneous part of an architecture system [3]. Furthermore, CGRAs have become highly important in HPC (High Performance Computing) systems, which greatly benefit from having heterogeneous-type PEs, with a mix of basic and complex PEs [11]. This is highly important for mathematical functions, since traditional CGRAs do not handle them optimally.

In the table I it is shown a comparison between four types of CGRAs. They present different trade-offs, where they are developed according to the area of application or workload. For example the HETA CGRA is good when the physical space is a concern [7] or the Plasticine is a good option when the workload is highly parallel [10].

Table I
Comparison of state-of-the-art solutions

Solution	Strengths	Weaknesses	Viability	PE / CGRA Type
HETA [7]	Bayesian DSE requires zero manual intervention, very fast CGRA.	Only functional and RTL simulation, limited veification tooling.	Good for research, ideal for temporal CGRA exploration.	Heterogeneous / Temporal.
TRAM [6]	High PE utilization and clean interconnect model.	Spatial CGRA only, cannot time-multiplex resources.	Good for research, when the interconnect flexibility and mapping are priorities.	Homogeneous / Spatial.
OpenCGRA [5]	Very complete, with multi-level simulation. Based on mature LLVM and PyMTL3 infrastructure.	The mapper can give a low PE utilization. Since it uses python-based modeling, large designs can be slower.	Ideal for research and prototyping, presenting a very complete verification and integration flow.	Homogeneous / Spatial.
Plasticine [10]	Massive per-cycle processing (OP/s), built for parallel patterns.	Rigid programming and restrictive power consumption.	Fixed infrastructure/servers, proved at 28nm with real performance numbers.	Heterogeneous / Spatial.

IV. Proposed Solution and Second-Level Architecture

Architectures like the homogeneous CGRA present a challenge where there is under-utilization of resources like PEs and interconnects [12], [13]. S ince all PEs are identical and have general-purpose capabilities, a large fraction of functional units remain idle during execution [7]. In addition, the uniform interconnects can leave many routes unused [3]. This kind of architecture has the advantage of being simple to design and implement, but that same simplicity is a limitation when high parallelism and throughput are required. Such architectures are not energy-efficient and have low performance [9].

In contrast, heterogeneous CGRAs have different types of PEs, which allows a better utilization of the resources like the HETA architecture [7], however even in this case still exists a limitation, where the PEs are not specialized enough and are limited to a specific workflow. For example the HETA architecture supports scalar operations but doesn't mention the support for vector operations [7], on the other hand, the Plasticine architecture supports vector operations but doesn't mention the support for scalar operations [10].

This work presents a heterogeneous CGRA with different PE types, routing cells and distributed memory cells. This design is proposed in order to have specialized cells according to the necessity of the workload, having a specialized element for a wider range of operations with less elements. The elements of the CGRA are as follows:

- **Routing:** FIFOs for dynamic, non-blocking flow.
- **Distributed memory:** Local SRAM banks (1–4 KB).
- **Scalar processing unit:** Multi-precision ALU (int8/16/32) + hardware loops.
- **Vector processing unit:** float32 SIMD (4–8 lanes) for up to 8 simultaneous elements.
- **MAC processing unit:** Multiply-Accumulate unit for high-throughput operations.

With these elements, we are introducing a new heterogeneous CGRA architecture that can be used for scalar operations and vector operations, with a better utilization of the resources and a better performance. Currently no other architecture has been found that combines scalar and vector operations, neither with routing cells and distributed memory cells, which makes this architecture unique in its kind.

A. Architectural Proposal

This architecture proposal is shown in the figura 1, where the different elements of the CGRA are present. In this case is a basic array of 2x2 but this could be expanded according to the necessity.

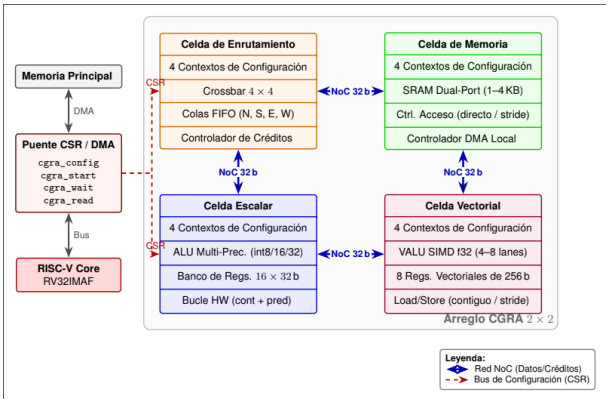

Figure 1. Second-level architecture diagram generated with artificial intelligence.

1) Routing Cell: First-class citizen (not static multiplexers): 4–8-entry FIFOs $\times$ 32 b per port; 4×4 crossbar (1 cycle, multicast); credit-based control; 4 contexts (1-cycle switch). Routing overhead $< 15\%$ area (vs. $\sim 30\%$ in HyCUBE [13]).

2) Distributed Memory Cell: 1–4 KB dual-port SRAM (FPGA BRAM); controller (direct/indirect/stride); NoC interface (1–2 cycles); local DMA (host-free block transfers). Eliminates centralized contention.

3) Scalar Cell: int8/16/32 ALU (4 int8 SIMD ops/cycle); 16 $\times$ 32 b registers; HW loops (counter + predication); 4 contexts; 32 b credit-based NoC interface.

4) Vector Cell: 4–8-lane float32 SIMD VALU (add, MAC, reduce); 8 vector registers (128/256 b); strided load/store; 4 contexts; 128 b NoC (burst mode). Will consume $\sim 3\times$ DSP, delivering $\sim 4\times$ float32 throughput [11].

5) NoC Router: 2D mesh, 32 b links; credits + 4-flit FIFO; round-robin arbitration; 1 cycle/hop; CSR/DMA host port (cell config., DMA transfers, result readout).

6) RISC-V Interface: Dedicated CSR (cell config., context, start/stop); 1-channel DMA; interrupts/flags; 4 custom instructions (cgra_config/start/wait/read). Reduces config. latency [3], [14].

B. Development Tools and Environment

The development of this heterogeneous CGRA starts with the implementation of SystemC to verify the functionality of the architecture and its elements. Afterwards, the SystemC model is translated so it can be used in Vitis 2024.1 and generate a RTL code and get the metrics of the design.

In order to ensure strict reproducibility of the design and simulation tests, the software tooling infrastructure and its respective versions are defined in the table II.

Table II
DEVELOPMENT TOOLS AND ENVIRONMENT VERSIONS

Tool / Software	Version	Purpose in the Experiment
SystemC	3.0.1	Modeling and simulation of the CGRA architecture functionality.
Vitis HLS	2024.1	High-Level Synthesis (HLS) from SystemC to RTL (VHDL).

V. EXPERIMENTAL PLAN AND METHODOLOGY

A. Testing and Stimulus

To test an electronic design, a testbench is needed, in charge of driving a stimulus into the design and facilitating monitoring of what happens in the design under test (DUT). As has been done in other designs, and in the various CGRA development frameworks, it becomes necessary to have an algorithm that the architecture is capable of executing. For this, one typically starts from so-called kernels: representative algorithms, typically coded in C (for example, Sum-Reduction and GEMM), annotated with compiler directives on their loops, whose data-flow structure can be mapped onto a CGRA's PE array. This is, in fact, the approach followed by Morpher within the OpenCGRA ecosystem: starting from the C kernel, it generates the data-flow graph (DFG) and the scratchpad memory layout, and produces the test vectors that feed the simulation/emulation stage, which are used to verify functional correctness and obtain area and power estimates [15]. The use of Sum-Reduction as a reference kernel is also consistent with the CGRA literature oriented toward signal processing, where it recurrently appears alongside routines such as IDCT, FIR, and IIR [2].

It is worth noting that even in well-established frameworks such as OpenCGRA, the authors themselves point out that the simulator relies on testbenches manually written by the user and does not automate test-data generation or end-to-end verification [15]. This reinforces the relevance of proposing, from the outset, a structured and progressive testing flow such as the one proposed below.

While a valuable goal is to achieve end-to-end execution of an algorithm such as Sum-Reduction or GEMM compiled from start to finish (from the C program to the configuration bitstream and the CGRA's input data), given the time available for the project's development, the first objective targets algorithmically simpler tests. The following steps are proposed:

1) **A single PE executing $c = a+b$ (constant) or $c = a \times b$.** This validates loading of the configuration word (config word), reading of registers/inputs, the ALU operation, and writing of the output result. It is, literally, the architecture's "hello world."
2) **Chains of operations, in pipeline mode:** accumulated sum ($y = x_0+x_1+x_2+\ldots+x_n$) and successive-squaring powers ($x,\ x^2,\ x^4,\ x^8,\ \ldots$).
3) **Simple MAC ($acc\ +=\ a \times b$):** this is the base block of almost every real kernel (Sum-Reduction, GEMM, convolutions), with this working correctly, the validation can scale toward more complex designs.

Once these basic tests are passed, progress can be made toward typical benchmarks from the CGRA literature, such as MachSuite and PolyBench [16], [17], which together with Wavelib and BEEBS make up the set of suites used by Morpher to evaluate its compilation flow [15]. These suites provide kernel libraries (including Sum-Reduction and GEMM) that

can be mapped onto the CGRA through a compilation flow similar to the one described above.

B. Monitoring

This project will be developed in SystemC, the aim is to leverage the language's own features to implement performance measurement. It is proposed to implement a monitor as an `SC_MODULE` component that observes, non-intrusively, DUT elements such as signals and internal counters. This separation between functional logic and measurement logic follows SystemC's own modular design principle, in which the monitor can be added to or removed from the testbench without modifying the module under analysis.

It is proposed that the monitor compute, at minimum, the following metrics:

- **Operations per second (OP/s):** ratio between the total number of operations executed by the DUT and the total execution time.
- **Average latency:** recording `sc_time_stamp()` at the start and end of each individual operation, averaged over the sampling window.
- **Utilization rate:** proportion of active cycles relative to the total number of cycles.

These metrics are printed to the console via `SC_REPORT_INFO`.

Additionally, once the design reaches a stage suitable for FPGA or ASIC synthesis, it will be possible to measure power, area, and *timing* metrics using the corresponding EDA tools. This applies both to the AMD Kria KV260 and the AMD Alveo U55C.

C. Architectural Exploration

Tied to design-space exploration, the plan is to generate variations of the CGRA's architectural design by modifying the number and type of PEs (*processing elements*). For example, changes in the distribution of PE types (more memory banks, more fixed-point ALUs, more floating-point ALUs), as well as different $n \times n$ array dimensions, aimed at optimizing the execution of vector instructions. This type of exploration is consistent with the diversity of CGRA architectures reported in the literature, where performance depends heavily on the composition and arrangement of the PEs [2].

D. Results

This section presents the results obtained from the implementation of the CGRA with the proposed kernels and monitoring metrics from section V.

1) Sum-Reduction Kernel: For the sum-reduction kernel we did two comparison, one temporal implementation with our new MAC cell and other spatial implementation with 4 scalar cells. Both of the solutions were implemented under the same hardware considerations, using a Kria K26 SoM as reference with a 250 MHz clock target. The results of these implementations are shown in the table III, where the new MAC cell shows a better utilization compared with the standard solution given by the literature, like the TRAM solution [6].

Table III
POST-SYNTHESIS RESOURCE UTILIZATION FOR SUM-REDUCTION KERNEL.

Resource	MAC Cell - Temporal	Scalar Cell - Spatial	Spatial/Temporal
LUT	1205	3857	3.20×
FF	1551	5230	3.37×
DSP	3	12	4.00×
SRL	0	20	–
BRAM	0	0	–

Regarding the performance metrics, the table IV shows that the temporal implementation with the MAC cell shows a better performance since this kind cell is designed to perform the sum-reduction operation in a more efficiente way that a normal scalar cell. With this results is possible to confirm the importance of having a heterogeneous CGRA with specialized cells for specific operations, since this allows to have a better performance and a better resource utilization.

Table IV
PERFORMANCE METRICS FROM MEASURED SUM-REDUCTION EXECUTION.

Metric	MAC Cell - Temporal	Scalar Cell - Spatial	Spatial/Temporal
Operations per second (OP/s)	19.63 MOP/s	17.97 MOP/s	0.92×
Average latency per operation	50.95 ns/op	55.65 ns/op	1.09×
Utilization rate (%)	79.7%	47.8%	0.60×

E. Preliminary Results

This subsection reports what has actually been measured against the plan of Sec. V, distinguishing completed results from work still in progress, consistent with the "preliminary" scope of this milestone.

1) Functional Verification: The table V summarizes the regression suite: 19 SystemC testbenches covering every module described in IV-A individually plus three integration levels (mesh, mesh wrapper/CSR-DMA, RISC-V-facing system), totaling 139 individual pass/fail assertions with **zero failures**. One additional testbench (MAC-cell sum-reduction) currently fails to *build* due to a missing standard-library header, a build-pipeline issue rather than a design defect; it is tracked as a known, easily-fixable gap rather than hidden from this count.

Table V
REGRESSION SUITE RESULTS (SYSTEMC TESTBENCHES).

Scope	TBs	Asserts	Result
Cell-level (scalar, vector, MAC, memory, routing)	12	79	12/12 PASS
Mesh integration (1×1, heterogeneous 2×2, smoke/complex)	4	51	4/4 PASS
CSR/DMA wrapper + sum-reduction dataflow	2	16	2/2 PASS
RISC-V-facing system integration	1	1	1/1 PASS

2) End-to-End Sum-Reduction Demonstration: The mesh's routing, memory, scalar, and vector cells jointly execute $total = seed + \sum_{i=0}^{6} v_i$, driven entirely through the RISC-V-facing CSR/DMA bridge (no direct cell access from the

testbench). Across three rounds with distinct signs and magnitudes, all outputs match the expected totals exactly. Counting the fixed-latency instruction-load/relay chain in the source (routing → memory → routing → scalar → vector, excluding the bounded DMA-completion poll), the dataflow completes in 27 fixed cycles plus at most 20 polling cycles (two single-element DMA transfers, each bounded at 10 cycles), i.e. $\leq$47 cycles per round at the simulated clock period — an upper bound derived from the source, not yet from an instrumented cycle counter.

3) HLS Cross-Validation: The arithmetic kernel of the sum-reduction benchmark was re-implemented as a standalone Vitis HLS C function and validated in C-simulation (`csim`) against the same three golden rounds used in the SystemC testbench, with an exact match on all outputs. This confirms the arithmetic contract is synthesis-tool-portable before committing to a full `csynth`/`cosim` run, whose resource and timing figures are not yet available and are deferred to the final report.

4) RISC-V Baseline Characterization: To contextualize the CGRA's expected benefit, the same sum-reduction kernel is being characterized as a compiled RISC-V (RV64GC) binary under gem5's system-call-emulation mode, comparing in-order (Atomic, Timing, Minor) and out-of-order (O3) CPU models via cycle counts in `stats.txt`. At the time of writing, the gem5 build is in progress; cycle-count results will be reported in the final article alongside the CGRA and HLS figures above.

AI Tool Usage Declaration

For this work, Artificial Intelligence was used to help structure the document, as well as for a grammar and spelling review of the document.

References

[1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.

[2] K. S. Artur Podobas and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE ACCESS*, vol. 8, 2020.

[3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "DVHetero: A framework for designing and validating heterogeneous SoC with RISC-V processor and CGRA," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 3, 2025.

[4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: https://www.edgeir.com//what-is-risc-v-and-why-is-it-important-20240109

[5] C. Tan, N. B. Agostini, J. Zhang, M. Minutoli, V. G. Castellana, C. Xie, T. Geng, A. Li, K. Barker, and A. Tumeo, "Opencgra: Democratizing coarse-grained reconfigurable arrays," in *2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2021, pp. 149–155.

[6] Y. Qiu, Y. Cao, Y. Dai, W. Yin, and L. Wang, "Tram: An open-source template-based reconfigurable architecture modeling framework," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022, pp. 61–69.

[7] Y. Dai, J. Li, Q. Zhu, Y. Qiu, Y. Hu, W. Yin, and L. Wang, "Heta: A heterogeneous temporal cgra modeling and design space exploration via bayesian optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 3, pp. 505–518, 2024.

[8] S. Das, K. J. M. Martin, and P. Coussy, "Context-memory aware mapping for energy efficient acceleration with cgras," *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a

[9] M. Wijtvliet, H. Corporaal, and A. Kumar, *Blocks, Towards Energy-efficient, Coarse-grained Reconfigurable Architectures*. Springer Nature Switzerland AG, 2022.

[10] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017.

[11] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, "Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, 2024.

[12] Mei, Bingfeng and Vernalde, Serge and Verkest, Diederik and De Man, Hugo and Lauwereins, Rudy , "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in , 2004, kU Leuven.

[13] Karunaratne, Manupa and Kulkarni, Aditi and Mitra, Tulika and Peh, Li-Shiuan , "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in , 2017, nTU Singapore.

[14] A. F. R. Ribeiro, "CGRA-RISC: Simulation Infrastructure for Coupling CGRA Accelerator to RISC-V Processor," Dissertation, Mestrado em Engenharia Inform'atica e Computaç ao, Faculdade de Engenharia da Universidade do Porto, Porto, Portugal, Jul. 2024, supervisors: Dr. Jo ao Paulo de Castro Canas Ferreira and Dr. Nuno Miguel Cardanha Paulino. Submitted July 24, 2024.

[15] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, "Building an open cgra ecosystem for agile innovation," in *Proceedings of the IEEE//ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9. [Online]. Available: https://arxiv.org//abs//2508.19090

[16] L.-N. Pouchet, "PolyBench/C: The polyhedral benchmark suite," https://www.cs.colostate.edu/~pouchet/software/polybench/, 2012, accessed: 2026-06-18.

[17] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "MachSuite: Benchmarks for accelerator design and customized architectures," https://github.com//breagen//MachSuite, 2014, gitHub repository. Accessed: 2026-06-18.